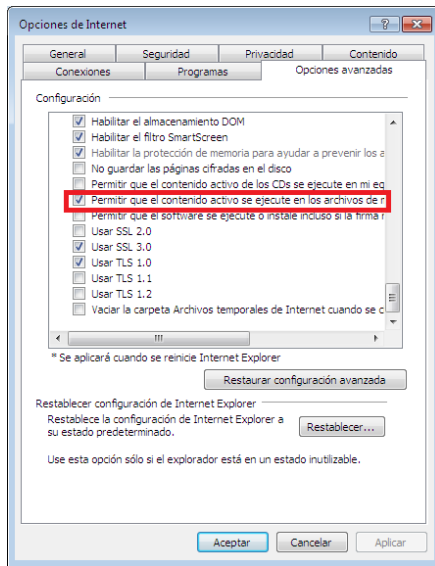


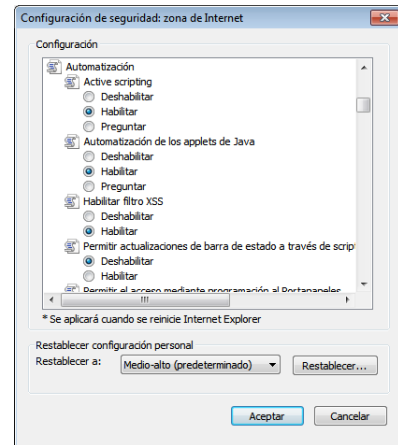
2. ASOCIACIÓN CON DOCUMENTOS XML (ISLA DE DATOS).



La funcionalidad del tema que nos ocupa sólo podremos ponerla en práctica en Internet Explorer 5 o superior y para ello debemos además tener activada la casilla que “permite ejecución de contenido activo” que se encuentra dentro de las propiedades del navegación, en la pestaña de “Opciones Avanzadas” como vemos en la imagen.

Además puede ser necesario permitir la ejecución de scripts de Java, si no está permitida, para lo que tendremos que entrar en la pestaña

de "seguridad" y poner un nivel bajo o medio de seguridad además de habilitar en la pestaña "nivel personalizado" la automatización de Java.



La asociación de datos nos permite mostrar documentos o información XML dentro de una página HTML convencional. Pero esto sólo es posible con aquellos documentos XML que estén estructurados simétricamente, igual que una base de datos típica. O sea, que sólo funcionaría si el documento tiene unos elementos que puedan interpretarse como registros y campos.

La asociación tiene inicialmente dos pasos principales:

1º) Vincular el documento XML a la página HTML en la que queremos mostrar los datos XML. Para ello usaremos la etiqueta <XML> también conocida como “*isla de datos*”. Por ejemplo:

```
<XML ID="inventario" SRC="inventario.xml"></XML>
```

donde el contenido del archivo inventario.xml sería por ejemplo:

```
<?xml version="1.0">
  <libro>
    <titulo> El Quijote </titulo>
    <autor> Miguel de Cervantes</autor>
    <paginas> 1500 </paginas>
    <precio> 20 </precio>
  </libro>
```

O también podríamos incluir el texto del archivo XML directamente como contenido del elemento <XML> del documento HTML:

```
<HTML>
<HEAD>
  <TITLE> Librería </TITLE>
</HEAD>
<BODY>
```

```

<XML ID="inventario">
  <?xml version="1.0">
    <libro>
      <titulo> El Quijote </titulo>
      <autor> Miguel de Cervantes</autor>
      <paginas> 1500 </paginas>
      <precio> 20 </precio>
    </libro>
  </XML>
  <!-- Aquí iría todo el resto del documento HTML -->
</BODY>
</HTML>

```

2º) Asociar los elementos HTML a elementos XML. Al realizar esta asociación, la etiqueta HTML mostrará automáticamente el contenido de su etiqueta asociada en el documento XML.

Esta asociación se puede realizar de dos maneras distintas:

- Asociación de datos de tabla, que permite asociar el elemento <TABLE> de HTML a los datos del documento XML, de modo que la tabla muestre automáticamente todos los registros del documento XML.

```
<TABLE DATASRC="#inventario"> </TABLE>
```

- Asociación de datos de un único registro, que permite asociar un elemento HTML que no sea una tabla. Por ejemplo, podemos asociar la etiqueta de HTML con la etiqueta <autor> de nuestro documento XML de esta forma:

```
<SPAN DATASRC="#inventario" DATAFLD="autor"> </SPAN>
```

de forma que sólo se represente un registro cada vez.

Veamos un ejemplo completo integrando todo lo anterior:

libreria.xml

```

<?xml version="1.0"?>
<inventario>
  <libro>
    <titulo> El Quijote </titulo>
    <autor> Miguel de Cervantes</autor>
    <paginas> 1500 </paginas>
    <precio> 20 </precio>
  </libro>
  <libro>
    <titulo> Viaje al centro de la Tierra</titulo>
    <autor> Julio Verne </autor>
    <paginas> 600 </paginas>
    <precio> 15 </precio>
  </libro>
</inventario>

```

libreria.html

```

<HTML>
  <HEAD>

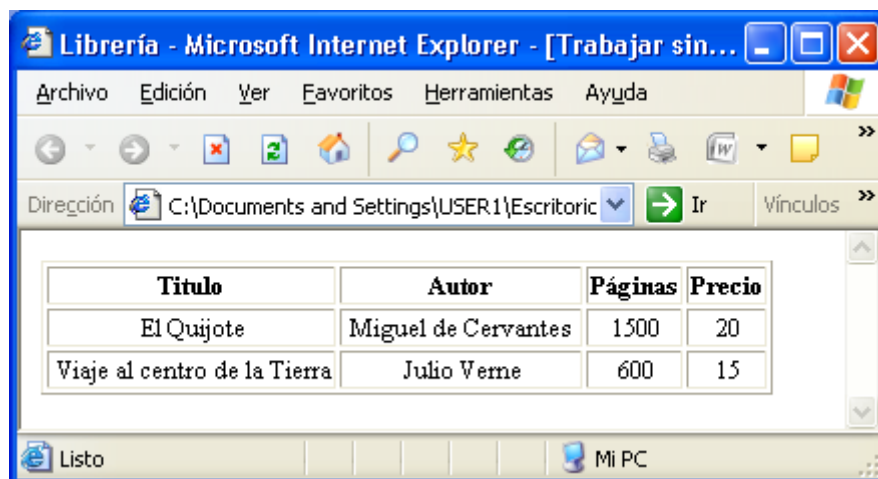
```

```

<TITLE> Librería </TITLE>
</HEAD>
<BODY>
  <XML ID="inventario" SRC="libreria.xml"></XML>
  <TABLE DATASRC="#inventario" border="1" cellpadding="5">
    <THEAD>
      <TH>Titulo</TH><TH>Autor</TH><TH>Páginas</TH><TH>Precio</TH>
    </THEAD>
    <TR align="center">
      <TD><SPAN DATAFLD="titulo"></SPAN></TD>
      <TD><SPAN DATAFLD="autor"></SPAN></TD>
      <TD><SPAN DATAFLD="paginas"></SPAN></TD>
      <TD><SPAN DATAFLD="precio"></SPAN></TD>
    </TR>
  </TABLE>
</BODY>
</HTML>

```

Cuyo resultado por pantalla sería este:



Automáticamente nos surgen dos preguntas, la primera es porqué no asociamos directamente el DATAFLD de cada elemento del documento XML al elemento <TD> de la tabla, ahorrándonos el tener que insertar un segundo elemento para esa asociación de campos. Y la segunda pregunta es, que si cada registro del documento XML tiene que ser representado en el documento HTML, cómo se realiza esa representación si sólo tenemos un registro <TR>, mientras que en el documento XML existen en este caso dos registros que corresponden a dos libros.

Las respuestas son las siguientes:

- el funcionamiento de la “asociación de datos” asegura que aunque el elemento <TABLE> defina una única fila, cuando el explorador represente la tabla, repetirá el elemento fila por cada registro del documento XML.
- Paradójicamente el elemento <TD> no es un elemento HTML asociable, en contra de lo que se pudiera pensar, por tanto debemos usar otro elemento asociable como

 para poder asociarle el campo del documento XML que nos interese. La lista de elementos HTML asociables es esta:

A, APPLET, BUTTON, DIV, FRAME, IFRAME,IMG,INPUT TYPE=CHECKBOX, INPUT TYPE=HIDDEN, INPUT=PASSWORD, INPUT TYPE=RADIO, INPUT TYPE=TEXT, LABEL, MARQUEE, SELECT, SPAN, TEXTAREA.

En general el número de registros suele ser grande y es necesario indicar el número de registros que queremos visualizar por página. Para ello usaremos el atributo DATAPAGESIZE que permite indicar el nº de registros por página. Por ejemplo:

<TABLE ID="inventariotabla" DATASRC="#inventario" DATAPAGESIZE="3">

Para navegar por las páginas de registros, podemos usar los métodos del elemento TABLE (respetando minúsculas y mayúsculas) que vemos en la siguiente tabla:

Método de TABLE	Efecto	Ejemplo
firstPage	Muestra la primera página	Inventariotabla.firstPage()
previousPage	Muestra la página anterior	Inventariotabla.previousPage ()
nextPage	Muestra la siguiente página	Inventariotabla.nextPage ()
lastPage	Muestra la última página	Inventariotabla.lastPage ()

Podemos invocar estos métodos de una forma muy sencilla, asignando el método al atributo ONCLICK de un elemento HTML BUTTON, DE ESTA FORMA:

<BUTTON ONCLICK="inventariotabla.nextPage()"> Siguiente página </BUTTON>

Modifiquemos y ampliemos el ejemplo anterior para observar estas opciones:

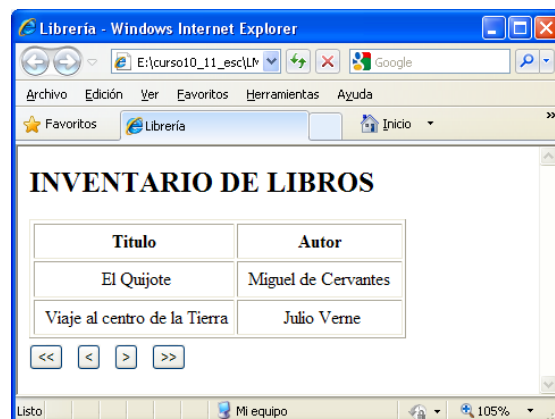
libreria.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<inventario>
  <libro>
    <titulo> El Quijote </titulo>
    <autor> Miguel de Cervantes</autor>
  </libro>
  <libro>
    <titulo> Viaje al centro de la Tierra</titulo>
    <autor> Julio Verne </autor>
  </libro>
  <libro>
    <titulo> Episodios Nacionales </titulo>
    <autor> Benito Perez Galdos </autor>
  </libro>
  <libro>
    <titulo> La familia de Pascual Duarte</titulo>
    <autor> Camilo Jose Cela </autor>
  </libro>
</inventario>
```

libreria.html

```
<HTML>
<HEAD>
<TITLE> Librería </TITLE>
</HEAD>
<BODY>
<XML ID="inventario" SRC="inventario.xml"></XML>
<H2>INVENTARIO DE LIBROS</H2>
<TABLE
  ID="inventariotabla" DATASRC="#inventario" DATAPAGESIZE="2"
  border="1" cellpadding="5">
<THEAD>
  <TH>Titulo</TH><TH>Autor</TH>
</THEAD>
<TR align="center">
  <TD><DIV DATAFLD="titulo"></DIV></TD>
  <TD><DIV DATAFLD="autor"></DIV></TD>
</TR>
</TABLE>

<Button onClick="inventariotabla.firstPage()">&nbsp;<<&nbsp;</Button>
&nbsp;<Button onClick="inventariotabla.previousPage()">&nbsp;<&nbsp;</Button>
&nbsp;<Button onClick="inventariotabla.nextPage()">&nbsp;>&nbsp;</Button>
&nbsp;<Button onClick="inventariotabla.lastPage()">&nbsp;>>&nbsp;</Button>
</BODY>
</HTML>
```



NOTA: Las islas de datos están diseñadas para funcionar en Explorer. Además en las versiones más modernas ya no se admite la asociación de datos XML. Es decir, a partir de Internet Explorer 10 ya no se admite.

El propósito es mejorar la interoperabilidad y la conformidad con HTML5. Esto significa que las islas de datos XML (asociación) ahora se analizan como HTML, al igual que en otros exploradores.

Una página que utiliza islas de datos XML (asociación) funcionaba correctamente en Windows Internet Explorer 9, pero ya no funciona en Internet Explorer 10 ni posteriores.

Para conseguir la funcionalidad anterior en Explorer 10 y posteriores, agregamos la siguiente etiqueta “meta” en el bloque de cabecera (HEAD) para optar por el comportamiento de Internet Explorer 9:

<meta http-equiv="X-UA-Compatible" content="IE=EmulateIE9">

Por tanto, el problema puede solucionarse rápidamente seleccionando el comportamiento de Internet Explorer 9, como se aprecia aquí:

```
<html>
  <meta http-equiv="X-UA-Compatible" content="IE=EmulateIE9">
  <xml>
    .....
  </xml>
</html>
```

Con vistas al futuro, la solución es revisar el código y eliminar la dependencia.

2. VALIDACIÓN. HERRAMIENTAS DE CREACIÓN Y VALIDACIÓN.

Como ya hemos indicado en este mismo tema y en otros anteriores, un documento XML puede ser “bien formado”, en cuyo caso cumple con las reglas sintácticas y de jerarquía impuestas por XML. Y si además, cumple con una serie de restricciones hechas a partir de un bloque de instrucciones o documento DTD, establecidas sobre los posibles elementos y atributos del documento, entonces podemos decir que el documento es además “válido”. El propio navegador que es un analizador, es capaz de indicar si el documento está bien formado, mostrándonos su estructura o si por el contrario no lo es, nos dará información de dónde se encuentran los errores detectados.

Los documentos XML se procesan a través de analizadores, aplicaciones que leen el documento, lo interpretan y generan una salida basada en sus contenidos y en la marca utilizada para su descripción. El resultado se muestra en un dispositivo de visualización, como una ventana de navegación o una impresora. Los procesadores hacen posible la presentación y distribución de documentos XML.

Los analizadores pueden tener o no la capacidad de validar: todos los analizadores comprueban que los documentos XML estén bien formados, pero solo los analizadores

validantes pueden efectuar la validación. Los analizadores no validantes procesan a mayor velocidad, ya que no tienen que comprobar tantos detalles como los validantes. Casi todos los procesadores de XML son capaces de validar con un esquema basado en DTD. Si el documento viola alguna de las reglas de su DTD, el analizador advertirá del error y el proceso del documento se detendrá. Un buen procesador validante no solo informará la presencia de un error, sino que elaborará un mensaje especificando cuál ha sido su causa. También existe el procesador Xerces de Apache que, además de ser el más completo parser de XML, incluye soporte de XML Schema.

Habitualmente, se usa un analizador validante para comprobar la corrección de los documentos, y uno no validante para su presentación. Una vez que se sabe que un documento es válido, no hay ningún motivo para comprobar su validez cada vez que se procese (optimizando así el tiempo del proceso).

Evidentemente, nuestro código ha de ser inspeccionado por un elemento externo, ya que nosotros mismos no podemos corregir aquello que en principio desconocemos como erróneo. En torno a un diseño XML, tenemos que decir, que las labores de validación de instancias de documentos XML, de DTD y esquemas XML, las de generación automática de DTD y esquemas XML a partir de un documento XML dado, o las de transformación de una DTD en un esquema XML, deben encomendarse a herramientas de validación y conversión que tenemos disponibles en la Web. Por ejemplo:

- <http://validator.w3.org/>
- Language Technology Group en la Universidad of Edinburgo validador basado en el RXP Parser
www.ltg.ed.ac.uk/~richard/xml-check.html
- Scholarly Technology Group en la Universidad Brown
www.stg.brown.edu/service/xmlvalid/
- Validación XML:
 - <http://www.w3.org/RDF/Validator/>
 - <http://www.xmlvalidation.com/>
 - <http://www.validome.org/xml/>
-
- expat, parser escrito en C por James Clark (<http://www.jclark.com/>)
- XML for Java (XML4J), IBM Alphaworks
(www.alphaworks.ibm.com/tech/xml4j)
- Lark, escrito en Java (www.textuality.com/Lark/)
- Apache Xerces (<http://www.apache.org/>)
- XP por James Clark (<http://www.jclark.com/>)
- Oracle XML Parser (technet.oracle.com/tech/xml)
- Sun Microsystems Project X (java.sun.com/products/xml)

3. DOCUMENTACIÓN DE ESPECIFICACIONES.

Podemos decir que XML no es un lenguaje, como se anticipó, sino una familia completa de tecnologías. La Recomendación o Especificación del W3C define la sintaxis y las etiquetas y atributos del lenguaje XML como tal, pero se le han ido agregando otros módulos opcionales con nuevas colecciones de etiquetas y atributos. Por ejemplo el estándar **XLink** para añadir enlaces de forma estándar a los documentos XML; **XSL** (para establecer hojas de estilo en XML). XSL se basa en **XSLT**, un lenguaje de transformación que sirve también fuera de XSL para añadir, borrar o reordenar etiquetas y atributos; **DOM**, un lenguaje que permite manipular archivos tanto

HTML <http://www.w3.org/DOM/> como XML <http://www.w3.org/TR/REC-DOM-Level-1/>. Como hemos visto anteriormente, existen también otras especificaciones como **XML Namespaces** (para asociar una URL con cada etiqueta y atributo de un documento XML), **XML Schemas 1** y **XML Schemas 2** etc.

Este sería un resumen de algunas de las más importantes especificaciones y lenguajes relacionados con XML:

Extensible Markup Language (XML) 1.0: especifica la sintaxis general de XML. Extensible Markup Language (XML) 1.0 (<http://www.w3.org/TR/REC-xml/>)

XML Linking Language XLL: tanto XPointer como XLink definen la forma estándar de generar enlaces entre recursos. (<http://www.w3.org/XML/Linking>)

XLink: describe cómo se asocian dos o más recursos. (<http://www.w3.org/TR/xlink/>)

Extensible Stylesheet Language (XSL): define la Hoja de estilos estándar para XML. (<http://www.w3.org/Style/XSL/>)

XSL Transformations (XSLT): (<http://www.w3.org/TR/xslt>)

XML Path Language (XPath) es un lenguaje para direccionar partes de un documento XML, diseñado para ser utilizado tanto por XSL como por XPointer.

(<http://www.w3.org/TR/xpath>)

DOM

DOM Nivel 1: (<http://www.w3.org/TR/REC-DOM-Level-1/>)

DOM Nivel 2 Core: (<http://www.w3.org/TR/DOM-Level-2-Core/>)

DOM Nivel 3 Core: <http://www.w3.org/TR/DOM-Level-3-Core/>

Namespaces in XML (<http://www.w3.org/TR/REC-xml-names/>)

XML Schema

XML Schema Part 0: Primer (<http://www.w3.org/TR/xmlschema-0/>)

XML Schema Part 1: Structures (<http://www.w3.org/TR/xmlschema-1/>)

XML Schema Part 2: Datatypes (<http://www.w3.org/TR/xmlschema-2/>)

RDF

.....

Por tanto, a la hora de construir documentos XML hay que tener en cuenta una serie de conceptos como: Espacios de Nombres (Namespace), XLink, DTD, XML Schema Definition (XSD), etc. y contar también con otros elementos para la presentación tales como las hojas de estilo CSS, XSLT, etc. o herramientas de procesamiento como DOM y SAX (SAX recorre secuencialmente los documentos XML, mientras que la tecnología DOM crea una estructura global en forma de árbol del documento XML para su recorrido). Por otra parte, existen lenguajes que lo complementan como XHTML, MathML, SVG, SMIL, etc. o que sirven para modificar aspectos semánticos con nuevos tipos de presentación de la información como RDF (Marco de Descripción de Recursos), OWL (Lenguaje de Ontologías Web), etc. Como ya dijimos al comenzar este tema, XML es un metalenguaje con estructura modular y una gran flexibilidad.

NOTA: RDF y OWL se usa en aplicaciones que necesitan procesar el contenido de la información y no sólo únicamente representarla. Aunque aparentemente RDF y OWL son similares, éste último posee un vocabulario mayor y podemos considerarlo más completo.